ENGINEERING FIELD GUIDE

LINUX SYSTEMS IN C

```
$ gcc -Wall -Wextra -pthread ptrunner.c
$ ./ptrunner --parallel tests/manifest.txt
[Worker 0] PASS: thread-safe design
[Worker 1] PASS: memory ownership
> THINK IN LIFETIMES, STATES AND INVARIANTS_
```

01 IO + C | 02 STREAMS | 03 PROCESSES | 04 THREADS

Practical models • annotated C • diagrams • traps • review drills

Grounded in Nour125666/linux-systems-engineering-lab • reviewed Nov 2026

How to use this guide

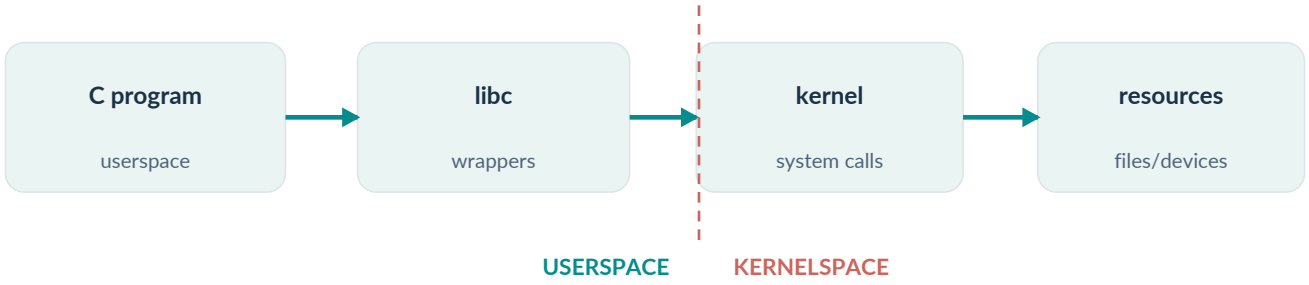
This is a readable engineering recap of your first four Linux Systems Engineering modules. It follows the actual exercises and project code published in your **Linux Systems Engineering Lab** GitHub repository, then adds carefully labeled explanations and sound practice patterns that help the ideas stick.

Read it with a terminal open. The diagrams give the model; the code proves it.

> **READING RHYTHM**
> Read a concept, sketch the diagram from memory, run or modify the tiny C example, and explain the failure path aloud. If you can predict a bug *before* running the code, you are starting to think like a systems engineer.

In the guide	What that means
FROM YOUR REPO	A concept or implementation directly represented by your checked-in exercises.
ENGINEERING EXTENSION	A recommended refinement or general POSIX/Linux explanation that goes beyond the current implementation.
WATCH OUT	A common trap, invariant, incorrect assumption, or failure path.
CHECKPOINT	A quick prompt you should be able to answer without Google.

The course in one page



Every later topic fits between your program, its memory and threads, the kernel, and the resources it controls.

Module	Hands-on repository anchor	Mental model
01 · Linux & C	copy.c, fd_offset_lab.c, bms_diag.c	File descriptors refer to kernel-managed open resources; byte streams must be validated.
02 · Streams & IPC	bms_service.c, frame_parser.c, crc8.c	A byte stream is not a message stream; event loops connect devices to clients.
03 · Processes	fork/exec labs, ProcessLab	A child is a separate process; ownership and shutdown require explicit lifecycle management.

Module	Hands-on repository anchor	Mental model
04 · Concurrency	ptrunner.c, qa_runner.sh	Threads share state; queues, locks, atomics and joins enforce invariants.

Table of contents

How to use this guide 2

MODULE 1 | Linux foundations 4

MODULE 2 | Streams and IPC 11

MODULE 3 | Processes and supervision 16

MODULE 4 | Threads and concurrency 21

The connections that make it click 29

Source map and further reading 33

Four questions to ask before you trust your code

> **A POCKET SYSTEMS-ENGINEERING COMPASS**
> **What state changes?** Read the variables and invariants. **Who owns the resource?** Track allocation, open and cleanup. **Who might be sleeping?** Name the wait condition. **What if this fails halfway?** Draw the shutdown path.

The fun part: the hardest bugs often come from one innocent assumption: "a read returns everything," "a wakeup means a job exists," or "fork shares my heap." Every chapter below replaces one of those assumptions with a reliable mental model.

Source snapshot: repository Nour125666/linux-systems-engineering-lab, branch **main**, audited 3 November 2026. Repo-derived specifics are attributed again in the source map at the end.

MODULE 01

Linux foundations

C memory, ELF, file descriptors and binary I/O

MODULE 1 | Linux foundations

Module 1 asked a deceptively simple question: **what actually happens when your C program touches a file or memory?** Your repository answers it through ELF inspection, an FD-offset experiment, a binary-safe copier, and an 8-byte telemetry decoder. These are the foundations under every serial driver, daemon, logger and debugging session that follows.

1. Linux is the boundary between a process and resources

An ordinary C application usually runs in **userspace**. The kernel controls physical memory mappings, scheduling, file systems, device interfaces and many kinds of I/O. A call such as **read()** enters the kernel through a system-call interface; the result is returned to the caller. The C library often provides the familiar user-facing wrapper. Not every library function is a system call: **strlen()**, for example, operates entirely in userspace.

FUNCTIONS YOU SHOULD RECOGNIZE

```
int open(const char *path, int flags, ...);
ssize_t read(int fd, void *buf, size_t count);
ssize_t write(int fd, const void *buf, size_t count);
int close(int fd);
off_t lseek(int fd, off_t offset, int whence);
```

> MENTAL MODEL
> A file descriptor is **not the file**. It is a process-local integer used to refer to a kernel-managed open resource. The kernel maintains the relationships and the current open state.

ELF, compiler, linker and loader

The pipeline is **source (.c) → compiler → object files (.o) → linker → ELF executable → loader → running process**. Your Week 1 environment notes used **readelf -l /usr/bin/ls** to inspect ELF program headers, PT_LOAD segments and the interpreter used for dynamically linked executables. Sections mainly organize information for linking and analysis; **program headers / segments** tell the loader which ranges must be mapped into the process.

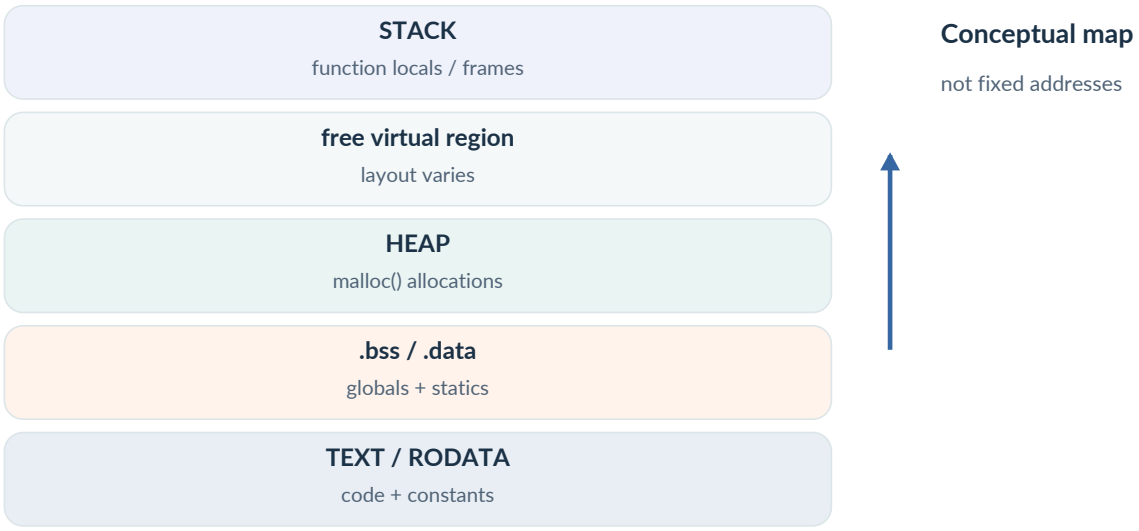
INSPECT A REAL EXECUTABLE

```
file /usr/bin/ls
readelf -h /usr/bin/ls
readelf -l /usr/bin/ls
nm -n ./program | head
cat /proc/self/maps | head
```

WATCH OUT

An ELF file is an executable format, not a snapshot of all runtime state. Stack contents, the exact heap layout and address randomization are runtime concerns.

2. Pointers, arrays, structures and memory regions



Conceptual userspace address-space regions. Real layouts vary with process, architecture, loader and ASLR.

Every pointer is a typed value that identifies an address. The type tells C how to interpret data at that address. **&x** obtains an address; ***p** dereferences a pointer; **p->field** is shorthand for **(*p).field**. The pointer itself has its own storage and lifetime, separate from the object it points to.

VALUES, POINTERS AND POINTER-TO-STRUCT

```
struct Sample { unsigned mv; int valid; };
struct Sample item = { .mv = 51200, .valid = 1 };
struct Sample *p = &item;

printf("%u\n", item.mv);     // direct field access
printf("%u\n", p->mv);       // pointer-to-struct
p->valid = 0;                // changes item.valid

unsigned *voltage_ptr = &item.mv;
*voltage_ptr += 100;         // changes item.mv
```

An array such as **uint8_t frame[8]** owns eight elements. In many expressions its name converts to a pointer to the first element, but an array and a pointer are not the same object. **sizeof(frame)** is eight within the scope where frame is an actual eight-byte array; in a function parameter declared **uint8_t frame[8]**, it behaves like a pointer parameter, so sizeof gives the pointer size.

POINTER PARAMETERS: THE CALLER MUST PROVIDE LIVE STORAGE

```
static void reset_sample(struct Sample *s)
{
    if (s == NULL) return;
    s->mv = 0;
    s->valid = 0;
}

reset_sample(&item);  // pass address; callee modifies item
```

> **WATCH OUT**
> **Pointer-to-struct safety checklist:** Is it NULL? Does it point to a still-live object? Is its type correct? May multiple threads access it? Who owns any nested allocation? The arrow operator answers none of these questions by itself.

Stack, heap, data and object lifetime

Automatic local variables commonly occupy stack-backed storage; global or static objects live throughout program execution; **malloc()** allocates dynamic storage that remains allocated until **free()**. A pointer can outlive the target it points to, creating a dangling pointer. Your **memory_lab.c** allocates an integer, checks for NULL, assigns through ***voltage**, prints it and frees it. That is exactly the right basic lifecycle.

ALLOCATION AND THE ONE-OWNER RULE

```
int *voltage = malloc(sizeof *voltage);
if (voltage == NULL) return EXIT_FAILURE;
*voltage = 3600;
printf("%d mV\n", *voltage);
free(voltage);
voltage = NULL;  // prevents reuse through this alias
```

- Use **malloc(sizeof *ptr)** to allocate space for the pointed-to type.
- A successful **malloc()** returns uninitialized storage; use **calloc()** when zero initialization is needed.
- **free(NULL)** is safe, but freeing the same live allocation twice is not.
- Never return the address of a local automatic variable after its lifetime has ended.
- **const** can express read-only access through an interface; it does not automatically resolve concurrent access or lifetime problems.

Pointers to pointers, argv and function boundaries

A pointer-to-pointer appears whenever a function needs to update a pointer owned by its caller. **char **argv** is a pointer to an array of argument-string pointers (with argv[argc] conventionally NULL). The POSIX **getline()** function accepts **char **lineptr** and **size_t *n** because it may allocate or resize the buffer and must give the caller the new pointer and capacity.

A POINTER-TO-POINTER CAN UPDATE THE CALLER'S POINTER

```
static int create_number(int **out)
{
    int *p = malloc(sizeof *p);
    if (!p) return -1;
    *p = 42;
    *out = p;     // changes caller's int *
    return 0;
}

int *value = NULL;
if (create_number(&value) == 0) {
    printf("%d\n", *value);
    free(value);
}
```

Your Week 1 **sensor.h / sensor.c / main.c** exercise also introduced a crucial build habit: declare function interfaces in headers with include guards, define them in one source file, and link translation units together. The **static** keyword at file scope limits external linkage; it is unrelated to the dynamic lifetime of a malloc allocation.

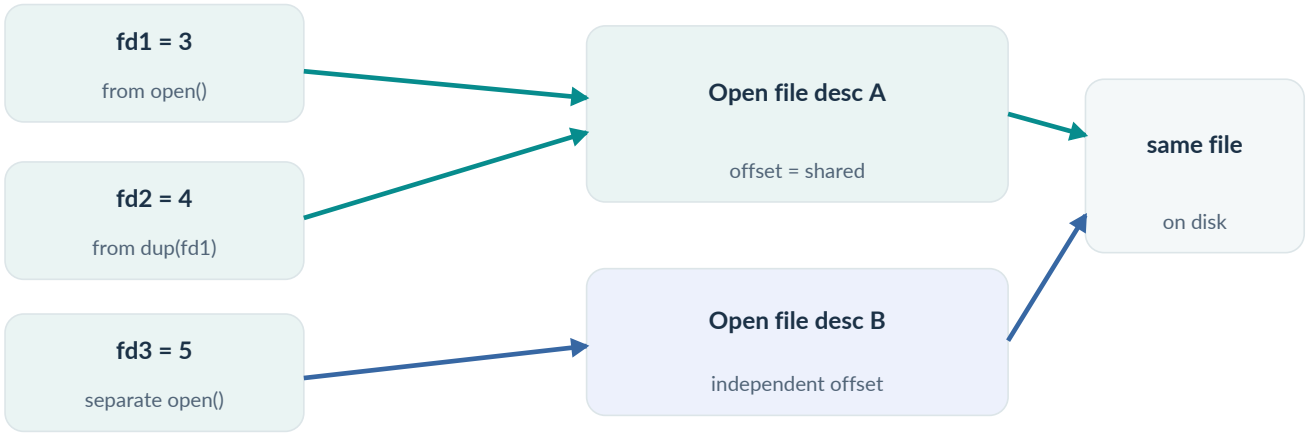
3. File descriptors are not filenames

A successful **open()** returns a nonnegative integer. Zero is a valid descriptor; only **-1** indicates failure. The lowest available FD may be reused after another descriptor is closed. Standard input, output and error are conventionally 0, 1 and 2, but those numbers are not permanently reserved. Your **test_copy.sh** explicitly tests that opening a file can return FD zero after stdin has been closed.

SAFE OPEN AND CLOSE PATTERN

```
int fd = open(path, O_RDONLY | O_CLOEXEC);
if (fd == -1) { perror("open"); return 1; }
/* operate on fd */
if (close(fd) == -1) { perror("close"); return 1; }
```

open() creates an *open file description* (the kernel state holding file offset and file-status flags). The process-level descriptor table holds references to such descriptions. **dup()** creates another descriptor that references the *same* description. A separate **open()** typically creates a new description, even for the same filename.



Modeled on your fd_offset_lab.c: fd1 and fd2 share an offset, while fd3 has an independent offset.

dup(), dup2() and lseek()

THE SHARED-OFFSET SURPRISE

```
int fd1 = open("data.bin", O_RDONLY);
int fd2 = dup(fd1);              // same open description
int fd3 = open("data.bin", O_RDONLY); // new description

char b[2];
read(fd1, b, 2);      // shared offset advances to 2
read(fd2, b, 2);      // reads from offset 2; now 4
read(fd3, b, 2);      // independent offset: now 2
lseek(fd2, 1, SEEK_SET); // fd1 and fd2 now see offset 1
```

lseek(fd, 0, SEEK_CUR) queries the current offset for seekable descriptors. **lseek()** is not meaningful on streams like pipes and many devices (it may fail with ESPIPE). **dup2(oldfd, STDOUT_FILENO)** is a standard building block for redirection. A duplicate shares open-file status flags and offsets, but close-on-exec is a descriptor flag with separate rules. Closing one duplicate does not automatically invalidate the other.

> INTERVIEW-GRADE DISTINCTION
> **Same path ≠ same open file description. Same open file description implies shared offset.** This distinction appears again when **fork()** inherits FDs in Module 3.

4. read() and write(): counts, interruptions, EOF

A successful **read()** may return fewer bytes than requested. This is normal on pipes, terminals, sockets and even some file operations. **read()==0** indicates end-of-file for a regular file, but interpreting zero on devices depends on the interface. **read()==-1** indicates an error and sets **errno**. A signal may interrupt a blocking call and produce **EINTR**.

COLLECT A FIXED-LENGTH BINARY FRAME

```
size_t got = 0;
while (got < sizeof frame) {
    ssize_t n = read(fd, frame + got, sizeof frame - got);
    if (n > 0) { got += (size_t)n; continue; }
    if (n == 0) break;       // EOF
    if (errno == EINTR) continue;
    perror("read");          // handle failure, then cleanup
    break;
}
/* got == 0: EOF between frames
   got == sizeof frame: complete
   otherwise: truncated frame */
```

A successful **write()** can also write fewer bytes than requested. The correct loop advances its source pointer by the number actually written. Your Week 1 **src/copy.c** handles partial writes, interrupted operations and a zero-byte write that makes no progress. The associated Bash suite checks missing paths, empty and binary files, /dev/full write errors and descriptor zero.

COMPLETE A WRITE SAFELY

```
size_t sent = 0;
while (sent < length) {
    ssize_t n = write(fd, bytes + sent, length - sent);
    if (n > 0) { sent += (size_t)n; continue; }
    if (n < 0 && errno == EINTR) continue;
    /* n == 0 or a real error: report, stop, clean up */
    break;
}
```

INVARIANT

Progress counters measure bytes successfully transferred, not bytes requested. For a binary protocol, completing one read() call does not imply completing one frame.

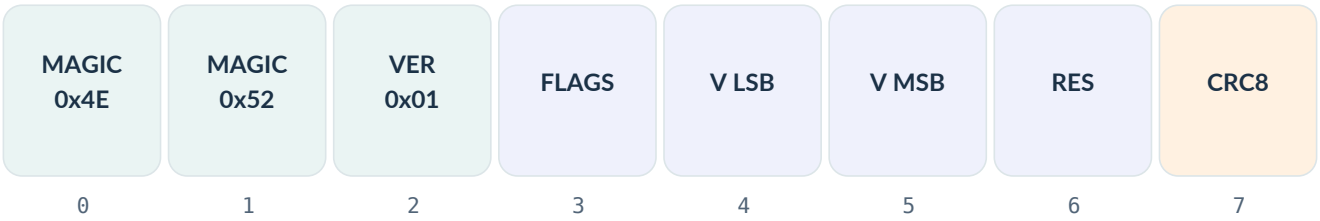
errno and error paths

For system calls that signal failure with -1, inspect errno promptly; a later library call may change it. Use **perror()** or **strerror(errno)** for diagnostics. Do not retry all errors blindly: retrying **EINTR** is often appropriate, while a nonblocking **EAGAIN** generally means wait for readiness. Treat every acquired resource as something that must be released on each exit path.

5. Binary telemetry: parsing is engineering, not casting

Your Module 1 **bms_diag.c** reads educational eight-byte telemetry frames from a file, validates two magic bytes and a version, checks for complete frames, and decodes a little-endian voltage. Module 1 uses reserved bytes 6–7; Module 2 evolves the format so byte 7 is CRC-8. That change matters: the two project protocols are related but **not identical**.

8-byte educational frame: indexes shown below each field



Module 2 packet layout shown here; in Module 1 bytes 6 and 7 were reserved rather than CRC-protected.

DECODE A LITTLE-ENDIAN 16-BIT VOLTAGE

```
unsigned voltage_mv =
    (unsigned)frame[4] | ((unsigned)frame[5] << 8);

/* Example: frame[4]=0x00, frame[5]=0xC8
   voltage = 0xC800 = 51200 mV = 51.2 V */
```

Endianness determines how multi-byte numeric fields are represented on the wire, not how bytes arrive over UART. Parse bytes explicitly instead of casting an arbitrary byte buffer to a C structure: structure padding, alignment, endianness and lifetime rules make raw casting error-prone. Frame length and protocol identity must be validated *before* interpreting payload fields.

- The **magic bytes** identify a candidate frame; they do not prove the payload is valid.
- The **version** determines which layout and interpretation are applicable.

- **Flags** are bit fields when the protocol defines them; use masks such as **flags & 0x04**, not assumptions about arbitrary bits.
- **EOF between frames** can be a normal stop; **EOF in the middle** is truncated input.
- Good tests include malformed headers, unsupported versions, incomplete frames, multiple frames and empty files.

Checkpoint • Module 1

1. Why does **dup(fd)** share the current offset but another **open()** normally does not?
2. A read requests eight bytes and returns three. What should the caller do next?
3. Explain what **struct Sample *p**, **&item**, ***p** and **p->mv** each mean.
4. Why can descriptor zero be valid? How does the code distinguish a genuine open failure?
5. What exact error should your binary telemetry reader report for an EOF after only five of eight frame bytes?

MODULE 02

Streams and IPC

Termios, poll, framing, CRC and Unix sockets

MODULE 2 | Streams and IPC

Module 2 moved from a file containing neat frames to the messier world of **live byte streams**. Your checked-in BMS service reads a virtual UART, finds valid frames among arbitrary bytes, remembers the latest valid voltage, and answers Unix-socket clients. This module is the bridge from a decoder to an actual Linux service.

1. A byte stream does not preserve your frame boundaries

Serial ports, pipes and stream sockets deliver a sequence of bytes. They do not promise that one writer call corresponds to one reader call. An 8-byte frame may arrive as 2+3+3 bytes, several frames may arrive in one read, or garbage may appear between frames. This is why your parser accepts **one byte at a time** and maintains state across calls.

> REMEMBER
> **The wire delivers bytes. The protocol invents messages.** Treat every read() result as an arbitrary chunk to feed into a persistent parser.

2. Virtual UARTs, termios and raw mode

The repository uses **socat** to connect two pseudo-terminals (PTYs), exposed through stable symlinks such as /tmp/ttyBMS0 and /tmp/ttyBMS1. One side sends simulated BMS bytes, while your Linux service opens the other side like a serial device. A PTY is a faithful lab interface for many terminal-I/O concepts, but it is not identical to a physical UART transceiver or its error characteristics.

CREATE A VIRTUAL SERIAL LINK

```
socat -d -d \
  PTY,raw,echo=0,link=/tmp/ttyBMS0 \
  PTY,raw,echo=0,link=/tmp/ttyBMS1
```

termios controls terminal behavior. Canonical mode generally collects line-oriented input; raw mode avoids line editing, echo and many special-character translations. In your code, **tcgetattr()** reads current settings, **cfmakeraw()** switches to raw behavior, and **tcsetattr()** applies the changes. Your integrated BMS service also selects **CS8, no parity, one stop bit** and a 115200 baud setting.

SERIAL CONFIGURATION CORE

```
struct termios tty;
if (tcgetattr(fd, &tty) == -1) /* error */;
cfmakeraw(&tty);
tty.c_cflag = (tty.c_cflag & ~CSIZE) | CS8;
tty.c_cflag &= ~(PARENB | CSTOPB);
cfsetispeed(&tty, B115200);
cfsetospeed(&tty, B115200);
if (tcsetattr(fd, TCSANOW, &tty) == -1) /* error */;
```

VMIN and VTIME: do not confuse them with poll()

Setting	Meaning in noncanonical mode
VMIN = 1, VTIME = 0	A blocking read normally waits until at least one byte is available, subject to other errors and signals.
VMIN = 0, VTIME = 0	Polling-style read: return available bytes immediately, potentially zero.
VMIN = 0, VTIME > 0	A timed read with a timer started when the read begins.
VMIN > 0, VTIME > 0	Inter-byte timeout behavior; actual rules are more nuanced.

Your serial reader also enables **O_NONBLOCK**. In nonblocking mode, the file-status flag can change read behavior independently of VMIN/VTIME; the call may return **EAGAIN** instead of waiting. Use **poll()** to avoid a CPU-burning retry loop. For the exact nuanced rules, consult termios(3) and the device behavior.

3. Event-driven I/O: poll() and nonblocking reads

Your **bms_service.c** monitors both the UART descriptor and a Unix-domain listening socket in one **poll()** call. This is an event loop: wait for readiness, inspect returned events, then perform the corresponding I/O. A single thread can coordinate multiple FDs without blocking forever on one idle source.

AN EVENT LOOP, SIMPLIFIED

```
struct pollfd fds[2] = {
    { .fd = uart_fd,   .events = POLLIN },
    { .fd = server_fd, .events = POLLIN }
};
for (;;) {
    int ready = poll(fds, 2, 1000);
    if (ready < 0 && errno == EINTR) continue;
    if (ready < 0) break;     // real error
    if (ready == 0) continue; // timeout, not failure
    if (fds[0].revents & POLLIN) receive_uart();
    if (fds[1].revents & POLLIN) serve_client();
}
```

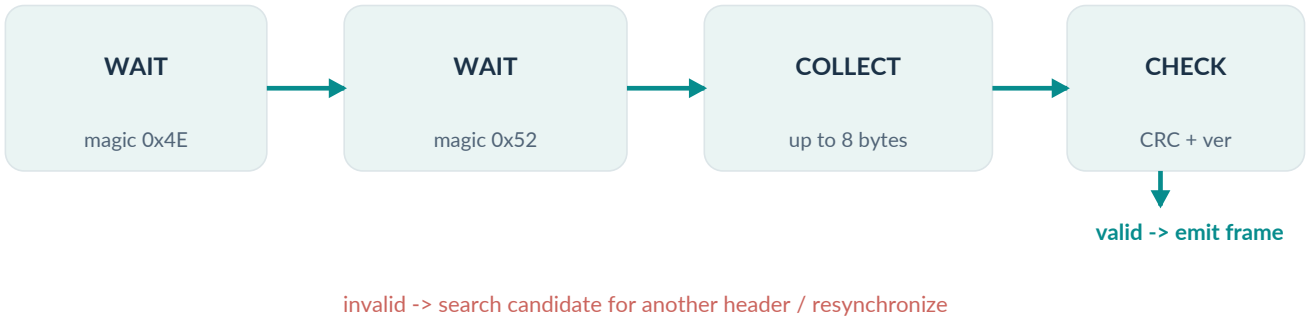
Condition	What it tells you
POLLIN	Reading is expected not to block for the indicated readiness condition.
POLLERR	Error state on a watched FD.
POLLHUP	A hangup or peer closure condition; there can sometimes still be buffered data to drain.
POLLNVAL	Invalid descriptor in the poll set.

Condition	What it tells you
poll() == 0	The timeout expired; not necessarily a hardware fault.
read(): EAGAIN	Nonblocking read currently has no data; wait/retry after readiness.

> **WATCH OUT**
> **Readiness is not a guarantee of an entire frame, a full socket message, or permanent availability.** Other activity can change state between readiness and I/O. Always inspect I/O return values.

4. Frame parser: explicit state beats wishful thinking

Your Module 2 parser defines three states: **WAIT_MAGIC_0**, **WAIT_MAGIC_1** and **COLLECT_FRAME**. It recognizes 0x4E, then 0x52, then collects enough bytes for an eight-byte candidate. Once the candidate is complete it checks the header, version and CRC. If validation fails, it searches the candidate for another possible header, preserving potential overlap to resynchronize after corruption or lost bytes.



A simplified view of frame_parser.c. In the real code, invalid candidates trigger header-search and state recovery.

STATE IS REMEMBERED ACROSS READ() CALLS

```
typedef enum {
    WAIT_MAGIC_0,
    WAIT_MAGIC_1,
    COLLECT_FRAME
} ParserState;

bool parser_push_byte(uint8_t byte,
                      uint8_t frame[8],
                      size_t *received,
                      ParserState *state);
```

Notice the pointer parameters: ***received** and ***state** are updated inside the parser, so state survives as the service feeds additional bytes. The parser owns no persistent heap allocations; the caller provides the frame buffer and persistent variables.

> **WATCH OUT**
> Do not simply skip eight bytes after every malformed candidate. The next valid header might begin *inside* the rejected bytes. Your parser intentionally searches for candidate headers to regain synchronization.

5. CRC-8: detect accidental corruption

Module 2 adds CRC-8 to byte 7. The code uses polynomial **0x07**, initial remainder **0x00**, and processes bytes 0 through 6. The emitted CRC byte must equal the calculated value. This is an integrity check for accidental corruption; it is **not** encryption, authentication or an anti-replay mechanism.

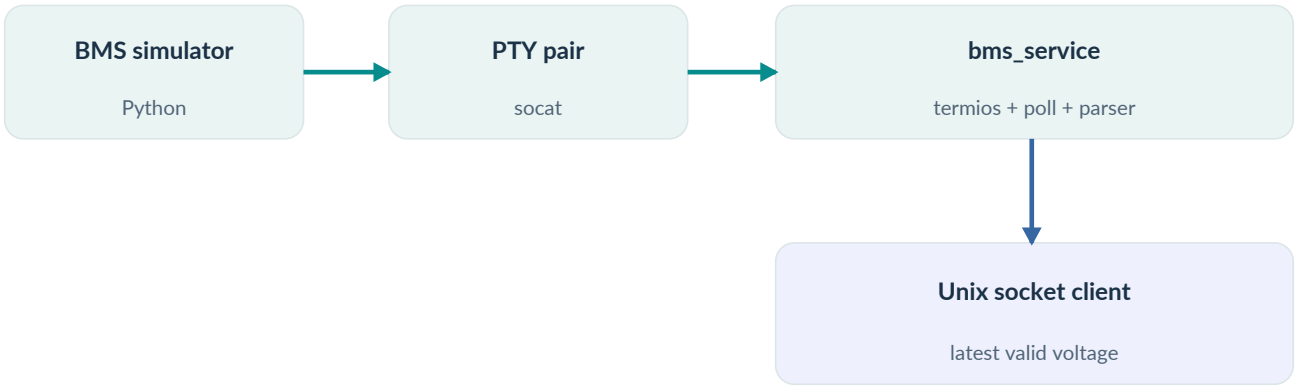
YOUR BITWISE CRC UPDATE PATTERN

```
uint8_t crc = 0x00;
for (size_t i = 0; i < length; ++i) {
    crc ^= data[i];
    for (unsigned b = 0; b < 8; ++b) {
        if (crc & 0x80)
            crc = (uint8_t)((crc << 1) ^ 0x07);
        else
            crc = (uint8_t)(crc << 1);
    }
}
```

Order of operations matters: frame detection, field validation, CRC verification and then accepting the decoded measurement. In your integrated service the parser only emits a **valid** frame; the latest reported voltage is updated from that accepted result. When corruption arrives, the previous valid voltage is preserved until a new valid frame is received.

6. Unix-domain sockets: processes communicating locally

The BMS service shares its most recent voltage through a pathname-based Unix-domain socket: **/tmp/bms.sock**. This is IPC between separate processes on the same host, not a network connection to an external machine. A stream socket gives an ordered byte stream; it still requires careful handling of partial sends and receives.



Your Module 2 architecture: PTY-based input, one poll-driven BMS service and a separate monitoring client.

SERVER LIFECYCLE

```
int s = socket(AF_UNIX, SOCK_STREAM, 0);
/* populate struct sockaddr_un */
unlink("/tmp/bms.sock");  // remove stale socket pathname
bind(s, (struct sockaddr *)&address, sizeof address);
listen(s, 5);
int client = accept(s, NULL, NULL);
/* send data to client, then close(client) */
close(s);
unlink("/tmp/bms.sock");
```

CLIENT LIFECYCLE

```
int fd = socket(AF_UNIX, SOCK_STREAM, 0);
connect(fd, (struct sockaddr *)&address, sizeof address);
ssize_t n = recv(fd, buf, sizeof(buf) - 1, 0);
if (n > 0) { buf[n] = '\0'; puts(buf); }
close(fd);
```

The **struct sockaddr_un** stores the AF_UNIX family and socket pathname. This is another useful example of pointers to structures: **&address** is the address of the object, and APIs expect a generic pointer to socket-address storage. In a durable service, verify path lengths, remove only owned socket files, handle client disconnects and write until the full response has been transferred.

> **PROJECT-SPECIFIC LIMITATION**
> Your current **bms_service.c** makes a single **send()** attempt for each client. It checks a negative return but does **not** fully handle a short positive send. It also does not yet implement a SIGINT/SIGTERM shutdown path or UART reconnection. These are documented limitations in the repository, not completed features.

7. Layering makes a service testable

You separated **crc8.c**, **frame_parser.c**, serial configuration and IPC demonstration code. That gives you test seams: the parser can be exercised with synthetic bytes without real hardware, and CRC vectors can be verified without a running socket server. The **test_frame_parser.c** suite checks garbage before frames, missing bytes, repeated magic sequences, corrupted input and resynchronization.

Layer	Owns	Should not assume
Transport	open/read/poll of PTY	That read returns complete frames
Parser	state + byte assembly	That each chunk is one message
Integrity	CRC + header validation	That CRC authenticates source
Application	latest validated voltage	That last-seen input is always valid
IPC	socket accept/send/recv	That one send always writes everything

Checkpoint • Module 2

1. A read returns 23 bytes, containing two complete frames and part of a third. How does your parser handle it?
2. Contrast **VMIN/VTIME** with **O_NONBLOCK + poll()**.
3. How can the UART and an IPC listening socket both be serviced in one thread?
4. What does CRC-8 detect, and what security property does it *not* provide?
5. What happens to the last validated voltage when a malformed frame arrives?

MODULE 03

Processes and supervision

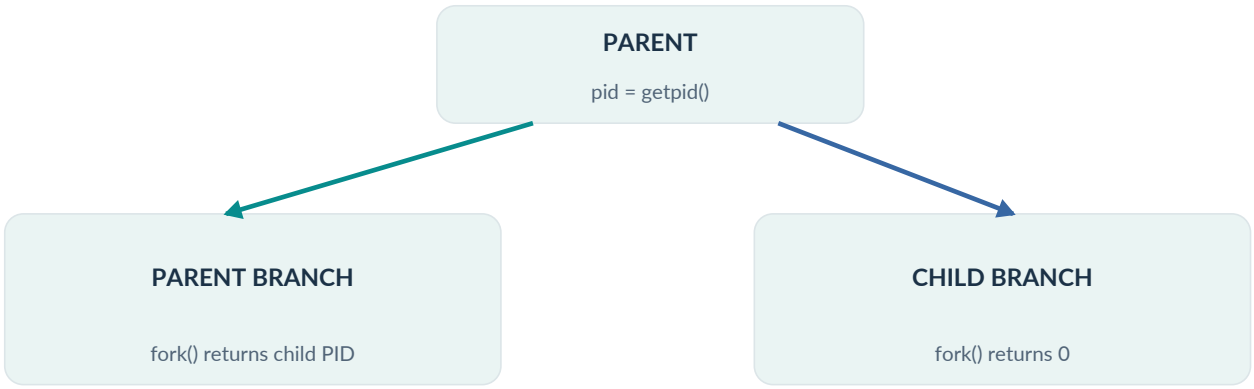
fork, exec, waitpid, signals and graceful shutdown

MODULE 3 | Processes and supervision

Module 3 turned one running program into a supervisor that launches, observes and stops another. Your **ProcessLab** is a single-child supervisor using fork/exec, waitpid(WNOHANG), sigaction, a five-second grace period, monotonic time and SIGKILL escalation. The hard part is not creating a process; it is **owning its entire lifetime**.

1. Process identity and the fork() split

A process has its own virtual address space and process identity, including a PID. **getpid()** identifies the caller; **getppid()** identifies the parent. **fork()** creates a new child process. Both processes resume after the fork call but observe different return values: -1 on failure, 0 in the child, and the child PID in the parent.



Separate process address spaces; inherited FDs can share open-file descriptions

Your fork_lab.c explores the parent/child return split and their independent PID identities.

FORK: ONE CALL, TWO EXECUTION PATHS

```
pid_t child = fork();
if (child == -1) { perror("fork"); return 1; }
if (child == 0) {
    /* child process */
    _exit(0);
}
/* parent process */
int status;
if (waitpid(child, &status, 0) == -1)
    perror("waitpid");
```

After fork, each process has its own address space. On modern Linux this normally uses copy-on-write memory mappings, so pages need not be copied immediately. Changes to an ordinary global, a stack variable or a heap allocation in the child do not change the parent's corresponding values. Your **fork_memory.c** explicitly explores these three storage kinds.

> **IMPORTANT DISTINCTION**
> Processes do **not** share ordinary global variables simply because one came from fork(). In contrast, threads within a process share the process address space. Shared-memory mappings are a deliberate separate mechanism.

2. File descriptors survive fork in a special way

The child inherits copies of the parent's file-descriptor table entries, and these entries often reference the **same open file descriptions** as the parent. That means an inherited descriptor can still share an underlying file offset or file-status flags across the process boundary. **Copy-on-write process memory** and **shared open-file description state** coexist; they are different mechanisms.

PREDICT THE SHARED FILE POSITION

```
int fd = open("log.txt", O_RDONLY);
pid_t pid = fork();
/* Assuming successful fork and suitable sequencing:
   both processes have their own fd integer entry,
   both entries refer to one shared open description. */
```

This is why file-offset experiments from Module 1 belong in your process mental model. Descriptor inheritance can also leak sensitive resources into an executed child unless you deliberately close descriptors or use **O_CLOEXEC / FD_CLOEXEC** appropriately. File descriptor ownership is a lifecycle problem, just like malloc ownership.

3. exec(): replacing a program, not creating a process

An **exec**-family call replaces the calling process's program image with a new executable. A successful exec does not return to the old instructions. The PID continues as the same process identity, but its code, address space and many runtime properties are replaced or reset according to POSIX/Linux rules. **execvp()** searches PATH for a program name; **execve()** accepts an explicit pathname and environment array.

CHILD LAUNCH WITH EXECVP()

```
pid_t pid = fork();
if (pid == 0) {
    char *const args[] = { "uname", "-a", NULL };
    execvp("uname", args);
    perror("execvp");
    _exit(127);  // executed only if exec failed
}
/* parent must wait/reap child */
```

Calling **_exit()** after failed exec avoids continuing into parent-oriented code and avoids the normal stdio flush and atexit work that may be inappropriate after fork. In your early fork/exec exercise you also explicitly flushed stdout before forking to avoid duplicated buffered output in child and parent.

Operation	Creates another process?	Replaces program image?	Typical use
fork()	Yes	No	Split parent and child execution.
execvp()	No	Yes, on success	Launch chosen executable in current process.
posix_spawn()	Yes	Child executes specified program	Create and execute with spawn-oriented interface.

Operation	Creates another process?	Replaces program image?	Typical use
pthread_create()	No (new thread)	No	Concurrent work inside one process.

4. waitpid(): no abandoned child lifecycle

Parent and child can execute independently. When a child terminates, the parent should collect its termination information with **waitpid()** (or a related API). Until reaped, a terminated child may remain as a **zombie** containing minimal status metadata. If its parent dies, an orphaned child is generally reparented to an appropriate reaper; the exact environment matters.

INSPECT A CHILD'S TERMINATION STATUS

```
int status;
pid_t r;
do { r = waitpid(child_pid, &status, 0); }
while (r == -1 && errno == EINTR);

if (r == child_pid) {
    if (WIFEXITED(status)) {
        printf("exit = %d\n", WEXITSTATUS(status));
    } else if (WIFSIGNALED(status)) {
        printf("signal = %d\n", WTERMSIG(status));
    }
}
```

WNOHANG makes waitpid return immediately if the requested child has not yet changed state. For the relevant child-status checks, a return of **0** means no waitable state is available yet, not that the child finished with exit status zero. Always distinguish the **waitpid return value** from the **status encoded in its output argument**.

> **WATCH OUT**
> Do not memorize signal numbers as a substitute for understanding. Ask: **Did the child exit normally?** If so inspect WEXITSTATUS. **Was it terminated by a signal?** If so inspect WTERMSIG. A shell's conventional 128+signal mapping is a presentation convention, not the raw wait status.

5. Signals: a request for attention, not a cleanup callback

A signal can interrupt or notify a process. **SIGINT** usually corresponds to Ctrl+C, **SIGTERM** requests termination, and **SIGKILL** cannot be caught, blocked or ignored. Use **sigaction()** to establish signal dispositions. A signal handler executes asynchronously with respect to the interrupted work, so it must do **very little**.

SMALL HANDLER, REAL CLEANUP IN NORMAL CODE

```
static volatile sig_atomic_t stop_requested = 0;

static void on_stop(int signo)
{
    (void)signo;
    stop_requested = 1;
}

/* Install using sigaction(SIGTERM, &sa, NULL)
   and sigaction(SIGINT, &sa, NULL). */
while (!stop_requested) {
    /* normal work */
}
/* close files, stop work, join/reap, then exit */
```

volatile sig_atomic_t is appropriate for a simple flag updated by a traditional asynchronous signal handler. It is **not** a general replacement for C11 atomics or mutexes used between threads. Avoid printf, malloc and pthread locking inside ordinary asynchronous signal handlers unless the documented async-signal-safety rules explicitly allow the operation; a simple flag makes cleanup occur from a predictable control flow.

The difference between graceful and forced shutdown

Graceful shutdown means making progress toward a known final state: stop accepting new work, allow or cancel work according to policy, close communications cleanly, collect child statuses and release resources. It does *not* mean hoping the process will exit eventually. Forced shutdown is an escalation when the graceful policy exceeds its deadline.

6. ProcessLab: the supervisor state machine



If child exits during grace: skip SIGKILL, still reap it

Your ProcessLab: SIGTERM request, five-second monotonic grace period, optional SIGKILL and eventual reaping.

The supervisor installs handlers, forks and execs the requested program, then periodically checks the child with **waitpid(..., WNOHANG)**. If a termination signal is received, the handler changes a flag. In normal control flow, ProcessLab sends **SIGTERM** to its child, starts a deadline using **CLOCK_MONOTONIC**, and sends SIGKILL if that child remains alive after the grace window. Finally, it reaps the child and reports its status.

SHUTDOWN LOGIC AS PSEUDOCODE

```
if (stop_requested && !term_sent) {
    send_SIGTERM(child);
    term_time = monotonic_now();
    term_sent = true;
}
if (term_sent && grace_period_expired() && !kill_sent) {
    send_SIGKILL(child);
    kill_sent = true;
}
/* keep checking waitpid(child, &status, WNOHANG) */
```

CLOCK_MONOTONIC is appropriate for measuring elapsed intervals because its progression is not affected by ordinary wall-clock adjustments. The included test child **ignore_term.c** lets you verify escalation rather than merely assuming SIGTERM will always work.

> **PROJECT-SPECIFIC LIMITATION**
> ProcessLab supervises **one child PID**, not arbitrary grandchildren or a full process group. Its README explicitly lists process-group supervision, health checks and automatic restart as future work. Terminating a shell PID does not necessarily terminate every subprocess it created.

7. Design choices and edge cases in a supervisor

- **Startup ordering:** Install required signal handling before fork to reduce unprotected startup windows. Decide which handlers or masks the child should reset before exec.
- **Interrupted waits:** If waitpid returns -1 with EINTR, retry rather than calling the child dead.
- **Child exit between checks:** A kill may return ESRCH if the target has already gone away; reconcile state with waitpid.
- **Exit reporting:** Separate failure of the supervisor itself from the child's test/program exit status.
- **Signals versus message queues:** A signal communicates an event, not a structured reliable command payload.
- **Process groups:** They become important if the launched application forks descendants you also need to stop.

8. Module 3 also prepares you for Module 4

Your Parallel Test Runner later replaced fork/exec in its worker-side launching code with **posix_spawn()**. The underlying lifecycle lesson stays unchanged: every launched child must be tracked, waited for and accounted for. The difference is that threads and processes share very different memory and synchronization properties. A pthread worker does not magically inherit responsibility for every child; the application deliberately assigns each worker a PID to wait for.

Checkpoint • Module 3

1. After fork, does a child modifying a heap integer also modify the parent's original heap integer? Explain.
2. How are fork return values different in parent and child?
3. Why is code after a successful execvp not executed?
4. What is a zombie process, and why should the supervisor call waitpid?
5. Explain TERM → grace deadline → KILL → reap, and why CLOCK_MONOTONIC is the right timer basis.

MODULE 04

Threads and concurrency

pthreads, mutexes, condition variables and worker pools

MODULE 4 | Threads and concurrency

Module 4 was the real integration challenge: take concepts you could understand separately, then make them coexist safely. Your **Parallel Test Runner** has a bounded queue, a main producer, two worker threads, atomic statistics, a subprocess launcher and shutdown logic. It exercises the key concurrency skill: **reasoning about every lifetime and every legal state transition**.

1. Threads share an address space; processes do not

Threads in the same process share globals, heap allocations, file descriptor table and process-wide resources, while each has its own execution context and stack. This makes communication easy but also creates data races. **pthread_create()** starts a thread running a function with signature **void *(*)(void *)**; **pthread_join()** waits for a particular joinable thread to terminate.

WORKER FUNCTION AND CONTEXT

```
typedef struct {
    unsigned worker_id;
    struct TestQueue *queue;
} WorkerContext;

static void *worker(void *arg)
{
    WorkerContext *ctx = arg;
    /* Use ctx->worker_id and ctx->queue */
    return NULL;
}
```

The **void *** argument can point to a context struct. It is the programmer's responsibility to ensure that this context remains alive until the worker no longer uses it. This is why your runner stores a separate **contexts[i]** for each worker and joins every successfully started thread before returning from the function holding those contexts.

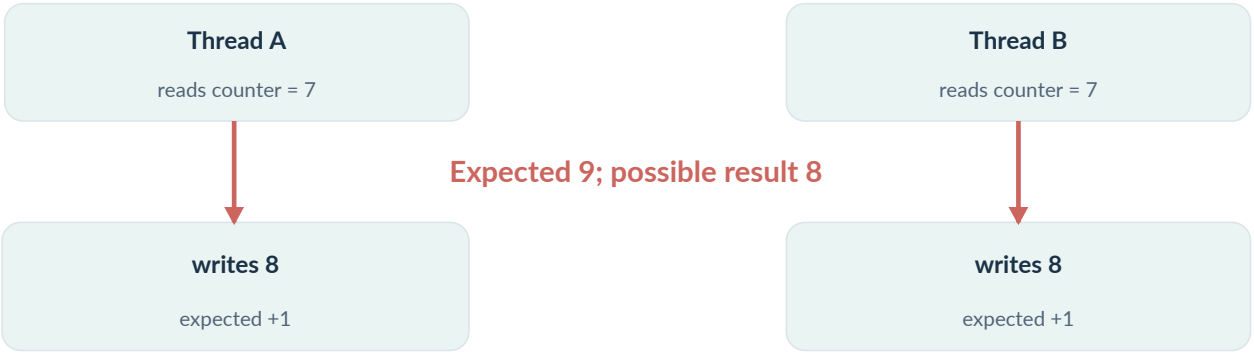
CREATION FIRST; JOINING LATER

```
size_t started = 0;
for (size_t i = 0; i < WORKERS; ++i) {
    contexts[i].worker_id = i;
    int rc = pthread_create(&threads[i], NULL,
                            worker, &contexts[i]);
    if (rc != 0) { /* close queue; join started */ break; }
    ++started;
}
/* after producer finished and queue closed */
for (size_t i = 0; i < started; ++i)
    pthread_join(threads[i], NULL);
```

> **WATCH OUT**
> Joining each new thread *inside* the same creation loop may serialize execution. Another classic bug is passing **&i** to every worker: they all observe one changing loop variable, not an immutable per-worker ID.

2. Data races: why counter++ is not safe

The expression **counter++** looks like one line but conceptually involves reading, computing a new value and writing it back. Two threads doing that to the same non-atomic object without synchronization create a **data race**, which is undefined behavior in C. Even on hardware with atomic aligned integer loads, the read-modify-write sequence is not automatically one atomic operation.



A lost update: two read-modify-write sequences overlap. The undefined behavior of a C data race is broader than just this illustration.

TWO SAFE APPROACHES (CHOOSE BY INVARIANT)

```
/* A) Protect a larger critical section */
pthread_mutex_lock(&stats_mutex);
processed++;
if (invalid) errors++;
pthread_mutex_unlock(&stats_mutex);

/* B) Independent atomic event counter */
atomic_fetch_add(&completed, 1u);
```

3. Atomics vs mutexes: the decision that matters

C11 atomics make specified operations on individual atomic objects safe from data races and provide memory-ordering semantics. A **mutex** establishes mutual exclusion around a critical section that may involve many fields, pointer structures or invariants. Neither is universally better. The problem to solve determines the tool.

Use case	Usually appropriate	Why
Increment an independent total	Atomic integer	One isolated counter update can be atomic.
Update head/tail/count and ring slots	Mutex	Several linked fields must change consistently.
Read a pair of logically related counters	Mutex or designed snapshot protocol	Two independent atomics can be individually race-free but not necessarily a consistent pair.

Use case	Usually appropriate	Why
Wake a consumer when work arrives	Mutex + condition variable	Wait on a state predicate without busy spinning.
Notify a signal handler to stop	volatile sig_atomic_t flag in traditional handler	Narrow async-signal-safe communication, not arbitrary interthread state.

> **ATOMIC ≠ TRANSACTIONAL**
> Suppose processed=10 and failed=2 are each atomic. Main can load processed just before a worker increments it and failed just after another update. Each load is safe, but the pair may not represent one coherent logical instant. If your report needs a consistent snapshot, design a shared synchronization protocol.

The default operations **atomic_load()** and **atomic_fetch_add()** use sequentially consistent ordering unless you explicitly request another memory order. Advanced memory-order choices belong later in the course; no special ordering is required for the simple independent completion counters in the runner. An atomic variable does *not* make surrounding non-atomic fields safe.

4. Mutexes: protect invariants, not entire workflows

A mutex guarantees exclusive entry to the critical section while it is held. In your queue, it protects **head, tail, count, closed** and buffer-slot ownership transitions. The protected invariant is conceptually **0 <= count <= capacity**, with head/tail indices valid modulo capacity.

KEEP EXTERNAL PROCESS EXECUTION OUTSIDE THE LOCK

```
pthread_mutex_lock(&q->mutex);
/* remove a command, update queue indexes */
pthread_mutex_unlock(&q->mutex);

int status = run_test(command); // may take seconds
atomic_fetch_add(&completed, 1u);
free(command);
```

If one worker held the queue mutex while waiting for an external test, other workers and the producer could be blocked for seconds. Critical sections should be **no longer than needed to preserve the shared invariant**. Still, do not shorten them by splitting a multi-field update into unprotected pieces.

Deadlock: two locks, incompatible order

If thread A acquires mutex 1 then waits for mutex 2, while thread B acquires mutex 2 then waits for mutex 1, neither can advance. Both are waiting while holding resources the other needs. A common remedy is **one global lock acquisition order** for all paths; release locks in a consistent, reasoned way after finishing protected work.

A SAFE LOCK-ORDERING RULE

```
/* Everywhere, always acquire mutex_a before mutex_b. */
pthread_mutex_lock(&mutex_a);
pthread_mutex_lock(&mutex_b);
/* shared operation */
pthread_mutex_unlock(&mutex_b);
pthread_mutex_unlock(&mutex_a);
```

5. Condition variables: wait on a predicate, not a promise

A condition variable is a way to **sleep until something may have changed**. It does not store the truth of the condition and does not protect memory by itself. A thread must hold the associated mutex, inspect a shared predicate, and use **pthread_cond_wait()** to atomically release the mutex and block. Before returning normally, the wait reacquires the mutex.

CORRECT EMPTY-QUEUE WAIT

```
pthread_mutex_lock(&q->mutex);
while (q->count == 0 && !q->closed) {
    pthread_cond_wait(&q->not_empty, &q->mutex);
}
if (q->count == 0 && q->closed) {
    pthread_mutex_unlock(&q->mutex);
    return false;
}
/* queue has at least one item; pop while holding lock */
```

Why while, not if? Wakeups can be spurious, and another consumer may remove the job after the signal but before this thread obtains the mutex. The awakened thread must evaluate the *current* predicate under the lock, not assume the notification reserved a job for it.

> **A ONE-SENTENCE MENTAL MODEL**
> A signal means **"please check the state again"**, not **"the state is definitely what you want"**.

signal() versus broadcast()

pthread_cond_signal() normally wakes at least one waiter, which is suitable when one inserted job can satisfy a consumer. **pthread_cond_broadcast()** wakes all waiting threads, which is important when closing the queue because both waiting producers and consumers must learn that the operating policy changed. Actual execution order after wakeup is a scheduling matter, not a promised FIFO wakeup order.

6. Bounded queue: the circular buffer and backpressure

Your runner uses an array of **eight char** * slots with **head**, **tail** and **count**. A push writes at tail then advances tail modulo capacity; a pop takes from head then advances head modulo capacity. Full means count == capacity; empty means count == 0. A bounded capacity prevents unlimited in-memory growth, but creates a necessary policy for what happens when the producer is faster than consumers.



One possible state of an eight-slot circular queue. Wrapped positions are normal; count disambiguates full from empty.

INDEX MOVEMENT: MODULO CAPACITY

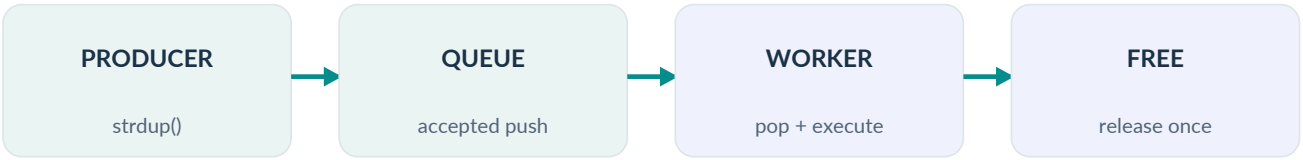
```
/* Protected by q->mutex */
q->commands[q->tail] = command;
q->tail = (q->tail + 1) % QUEUE_CAPACITY;
q->count++;

/* Later, on pop: */
*command = q->commands[q->head];
q->head = (q->head + 1) % QUEUE_CAPACITY;
q->count--;
```

A producer facing a full queue can block on **not_full** (as your runner does), drop data under an explicit policy, or divert it to another storage mechanism. None of these makes an overloaded system magically infinitely capable. For a real-time CAN capture task, blocking the receive thread may allow the kernel/hardware receive buffers to overflow; you must budget queue capacity, throughput and acceptable loss.

7. Memory ownership across the thread boundary

The main thread uses **getline()** to read a manifest, then **strdup()** to allocate an independent string for each accepted job. Why? getline reuses or reallocates its line buffer; handing workers the same **line** pointer would expose changing or freed storage. A successful queue push transfers responsibility for that particular allocation to the consumer; the worker frees it once execution finishes.



Rejected push? Producer frees the copy. Successful push? Worker frees it.

Ownership is transferred, not duplicated, at a successful queue push and pop.

THE FULL THREE-WAY OWNERSHIP CONTRACT

```
char *copy = strdup(line);
if (copy == NULL) {
    /* allocation failed: nothing to enqueue or free */
    report_failure();
} else if (!queue_push(&queue, copy)) {
    free(copy);   // rejected: producer still owns copy
    report_failure();
} else {
    /* accepted: worker will eventually free copy */
}
```

WATCH OUT

If you enqueue the one reusable **getline()** buffer many times, the jobs may all point to the same memory. If producer and worker both free an accepted job, you get a double-free. Ownership is as important to correctness as locking.

8. Queue closure: graceful shutdown is a protocol

Closing the queue sets a shared flag under the mutex and broadcasts to waiting threads. It means **no new jobs may be accepted**, not **discard every existing job**. Your consumer code returns false when the queue is both closed *and* empty, so it drains accepted jobs before terminating. A blocked producer wakes, rechecks closed and rejects its still-owned command rather than inserting after shutdown.

State	queue_push()	queue_pop()
Open, empty	Can insert	Waits for not_empty
Open, partially filled	Can insert	Removes a job
Open, full	Waits for not_full	Removes a job
Closed, nonempty	Rejects new job	Drains remaining jobs
Closed, empty	Rejects new job	Returns false / worker exits

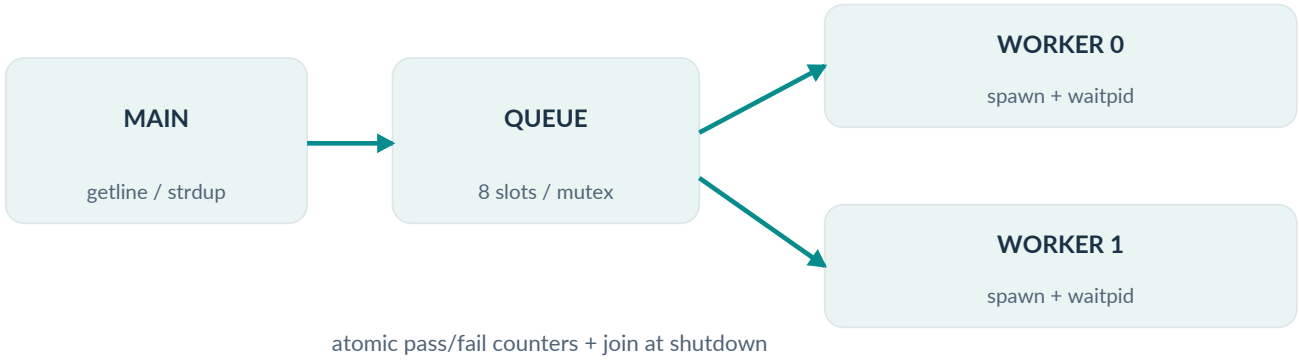
SHUTDOWN PROTOCOL

```
pthread_mutex_lock(&q->mutex);
q->closed = true;
pthread_cond_broadcast(&q->not_empty);
pthread_cond_broadcast(&q->not_full);
pthread_mutex_unlock(&q->mutex);

/* Main then joins all successfully started workers. */
```

This is different from canceling already-running children. Your current runner deliberately waits for each external test to finish, and its README warns that a hung command can prevent termination. A future tool could enforce per-test timeouts and manage child process groups, but that is outside the Module 4 implementation.

9. Thread pools and child processes work together



Your runner architecture: one producer, a bounded queue, two worker contexts and per-test child processes.

The workers run **run_test()** on dequeued commands. That function creates a shell child with **posix_spawn()**, waits for its own child PID using **waitpid()**, then returns a normalized outcome. Workers atomically increment passed or failed and free each command. The main thread closes the queue, joins all started workers and checks that **completed == enqueued**.

CHILD PROCESS STATUS, SIMPLIFIED

```
pid_t child;
char *const args[] = { "sh", "-c", command, NULL };
int rc = posix_spawn(&child, "/bin/sh",
                     NULL, NULL, args, environ);
if (rc != 0) { /* rc is an error number */ }
else {
    int status;
    while (waitpid(child, &status, 0) == -1 &&
           errno == EINTR) { }
    /* Inspect WIFEXITED / WIFSIGNALED */
}
```

The fact that two threads are waiting in waitpid does not make them wait for the same child: each uses the PID returned by its own spawn. Their child programs are separate processes; their output may interleave on the terminal because both inherit the same stdout/stderr destinations.

10. Startup and cleanup must survive partial failure

Suppose worker 0 starts and worker 1 creation fails. Returning immediately would leave worker 0 waiting on a queue stored in a soon-to-disappear stack frame. The correct lifecycle is: record how many threads started, stop producing, **close the queue to wake waiters**, join each started worker, then destroy synchronization objects once no thread can access them.

FAILURE-PROOF OUTLINE (NORMAL JOIN PATH)

```
size_t started = 0;
for (size_t i = 0; i < WORKERS; ++i) {
    int rc = pthread_create(&workers[i], NULL,
                            worker_fn, &contexts[i]);
    if (rc != 0) { success = false; break; }
    ++started;
}
if (success) produce_manifest_jobs();
queue_close(&queue);
for (size_t i = 0; i < started; ++i)
    pthread_join(workers[i], NULL);
/* only then destroy queue synchronization objects */
```

> **RESOURCE-LIFETIME RULE**
> The stack-allocated queue and worker contexts must outlive every worker that can reference them. Failed **pthread_join()** recovery is a harder case: your current project intentionally does not claim production-grade recovery for such rare failures.

11. Testing concurrency: evidence, not magical proof

Your project includes **--queue-selftest** (12 jobs), passing/mixed/empty manifests, a 40-job wraparound test, repeated queue tests, ASan/UBSan and reported ThreadSanitizer checks. The observed WSL timing of four independent one-second sleeps was **4.044 s sequential** versus **2.019 s parallel**: a speedup of about **4.044 / 2.019 = 2.00×** for this wait-heavy example.

Tool/check	What it can show	What it does not prove
ASan	Observed invalid memory accesses / some leaks with leak detection	No possible future memory bug

Tool/check	What it can show	What it does not prove
UBSan	Observed specified undefined operations under instrumentation	Absence of all undefined behavior
TSan	Observed data races on instrumented paths	No deadlocks or unexecuted races
100 iterations	Repeatability over sampled schedules	All possible schedules are safe
40-job wraparound	Ring indices survive more jobs than capacity	No starvation, timeout or cancellation issue
sleep benchmark	Overlapping waits improve elapsed time	Same speedup on CPU-bound code

Running tests in parallel introduces another problem: **test isolation**. Two tests writing the same temporary filename, binding the same port or changing shared configuration can interfere even when the test runner itself is perfectly synchronized. Use unique temporary directories, per-test environments or explicit serialization for shared resources.

Checkpoint • Module 4

1. Why should pthread_join usually be in a separate loop after creation?
2. Can two atomic counters alone guarantee a logically consistent snapshot? Explain.
3. Why does a condition-variable waiter recheck a predicate with while?
4. What should a closed-but-nonempty queue do on pop versus push?
5. Describe every command-string owner from strdup() to free().
6. What should main do if the second of two worker creations fails?
7. Why does the measured twofold sleep speedup not imply twofold throughput for CPU-bound tests?

The connections that make it click

The modules are not four unrelated courses. They describe **one system** seen from four angles: bytes, resources, processes and concurrent work. The best engineers are able to move between these views without losing track of who owns a resource or which state transitions are legal.

An end-to-end engineering scenario

Imagine your Linux application receives an eight-byte BMS frame, validates it, stores the latest voltage, logs measurements and serves a monitoring client. Here is what you would recognize now:

Layer	Question to ask	Relevant skill
Bytes	Did this read provide a full frame?	Partial I/O + persistent parser
Protocol	Is the candidate intact and in the expected version?	Magic/version/CRC + resync
Resources	Which file descriptors exist; who closes them?	open/close, file/socket lifecycle
Scheduling	Which descriptors are ready for service?	O_NONBLOCK + poll
Concurrency	Can the logger block the receive path?	Queue + threads + backpressure
Memory	Who owns the received frame or copied command?	Pointers, heap lifetime, ownership
Processes	Who created the child, who waits for it?	fork/exec/spawn/waitpid
Shutdown	How do we stop accepting work and finish safely?	Signal, close, wake, join, reap

Four words to ask during every code review

1 — STATE
What variables encode the condition of the system? Queue count? Parser state? Child PID? stop_requested? What invariants must remain true?

2 — OWNER
Who is responsible for every descriptor, child, thread, allocation and pointer? When does that responsibility transfer?

3 — WAITER
Who could be blocked right now? What precise event wakes them? What predicate must they recheck after waking?

4 — FAILURE
What if this call returns early, is interrupted, allocates nothing, writes less than requested or fails after half the setup succeeded?

One-page syscall and pthread compass

API	Core interpretation	Success / failure clue
open()	Obtain an FD referring to an open file description	FD >= 0 / -1 + errno
read()	Move bytes into user memory	0 = EOF; positive = count; -1 = error
write()	Move bytes out; may be partial	Positive count / -1 + errno
lseek()	Change/query offset of seekable resource	Resulting offset / -1
dup()	New FD referring to same open description	New FD / -1
poll()	Wait for descriptor readiness	Ready count / 0 timeout / -1
fork()	Create child process	0 child / PID parent / -1 failure
execvp()	Replace caller's program on success	No return on success / -1 failure
waitpid()	Collect a particular child state	PID / 0 with WNOHANG / -1
pthread_create()	Start a new thread	0 / nonzero error code
pthread_join()	Wait for a joinable thread	0 / nonzero error code
pthread_cond_wait()	Release mutex, wait, reacquire	Returns with mutex held on normal return
posix_spawn()	Create child and execute program	0 / nonzero error code

TWO ERROR CONVENTIONS

Many system calls report **-1 and set errno**. Pthread APIs and posix_spawn typically return an **error number directly** instead. For the latter, use **strerror(rc)** rather than assuming errno contains the failure.

Classic traps you can now recognize

Trap	Why it breaks	Replacement habit
fd > 0 means success	Descriptor 0 is valid	Check fd == -1 only
read == requested bytes	Short reads are legal	Track a byte count
one read = one frame	Streams do not preserve packets	Persistent parser state
same filename = same offset	Each open creates a new description	Track open file descriptions
fork shares ordinary heap writes	Processes have separate address spaces	Model COW and explicit shared memory
exec creates another process	exec replaces caller	fork + exec or posix_spawn
signal handler cleans everything	Not all functions are async-signal-safe	Set flag; clean in normal flow
counter++ is one atomic step	It is a read-modify-write	Use atomics or mutex
cond_signal means job reserved	Wakeup does not guarantee predicate	Recheck under mutex with while
atomic means transaction	Independent atomics do not lock a group	Protect multi-field invariants
queue close drops accepted work	Closed + nonempty can still drain	Define closure policy explicitly
tests passed = no races possible	Finite execution coverage	Combine tools, review and stress

Five tiny tracing exercises: predict before you run

Exercise A: pointers and structures

WHAT GETS PRINTED?

```
struct Item { int value; };
struct Item x = {.value = 3};
struct Item *p = &x;
p->value += 4;
printf("%d\n", x.value);
```

Exercise B: the descriptor-table surprise

ASSUME A SEEKABLE FILE CONTAINS ABCDEF

```
int a = open("letters.txt", O_RDONLY);
int b = dup(a);
char ch;
read(a, &ch, 1); // A
read(b, &ch, 1); // what character?
/* What offset do a and b now share? */
```

Exercise C: a queue that is already closed

STATE BEFORE POP()

```
q.closed = true;
q.count = 2;
/* What happens on three sequential queue_pop calls? */
```

Exercise D: child status

ASSUME THE CHILD EXITS NORMALLY WITH STATUS 7

```
int status;
waitpid(pid, &status, 0);
/* Is status itself equal to 7?
   Which macro extracts the exit code? */
```

Exercise E: overloaded logger

A CAN receiver produces 1,000 frames/s and a logger processes 600 frames/s. With 800 free queue slots and constant rates, how long until those slots are consumed? Is blocking the receiver always acceptable?

Exercise answers and engineering reasoning

A. 7. The pointer targets the same live structure object; p->value modifies x.value. **B.** The second read sees B; both descriptors now share offset 2 because dup referenced the same open file description. **C.** The first two pops return jobs and reduce count to zero; the third returns false. Closure rejects pushes, not already accepted jobs. **D.** Do not compare the raw wait status directly to 7; verify WIFEXITED(status), then use WEXITSTATUS(status). **E.** Backlog grows at 400 frames/s; 800 / 400 = 2 seconds. Blocking the receiver may cause an upstream overflow, so you need a deliberate loss, buffering or throughput policy.

A compact fault-investigation playbook

- **1. Reproduce:** Write the smallest command and input that trigger the failure. Preserve exit status and stderr.
- **2. Observe:** Is the symptom a wrong value, a block, a crash, a leak, unexpected child status or corrupt output?
- **3. Locate:** Identify the operation and shared state at the failure boundary. Distinguish caller error from API failure.
- **4. Hypothesize:** Write one falsifiable explanation: "two workers share a changing pointer" is stronger than "threads are buggy".
- **5. Test:** Use logs, controlled inputs, repeated runs, strace, GDB and sanitizers when appropriate.
- **6. Fix & regress:** Make the smallest correct change and add a test that would have caught the original bug.

> **BRIDGE TO MODULE 5**
> You have now written code that can **fail** in interesting ways. Next you will become fluent at **proving why**. That is the purpose of debugging, profiling, tracing and root-cause analysis.

Source map and further reading

The sections above synthesize the checked-in repository and stable POSIX/Linux behavior. The repository links below provide the exact student implementations; the manual pages provide API contracts and caveats. Code snippets in this guide are deliberately shortened or pedagogically adapted rather than copied wholesale.

Your GitHub repository — audited files

Root overview — README.md
Course goals and environment

M1: copy and partial I/O — month-01/week-01/src/copy.c
read/write, EINTR, short writes

M1: FD offsets — month-01/week-01/day02/fd_offset_lab.c
dup versus open, lseek

M1: ELF environment — month-01/week-01/environment.md
readelf, /proc/PID/fd

M1: telemetry decoder — month-01/module-01-project/bms_diag.c
8-byte file protocol

M1: regression suite — month-01/week-01/tests/test_copy.sh
failure and success tests

M2: project specification — month-02/module-02-project/README.md
PTY / UART / service architecture

M2: serial reader — month-02/module-02-project/serial_reader.c
raw mode, poll, nonblocking I/O

M2: parser — month-02/module-02-project/frame_parser.c
state, resync, CRC

M2: CRC — month-02/module-02-project/crc8.c
polynomial 0x07

M2: integrated BMS service — month-02/module-02-project/bms_service.c
poll and Unix socket

M3: process memory — month-03/module-03-processes/fork_memory.c
fork and separate state

M3: exec exercise — month-03/module-03-processes/exec_lab.c
execvp and waitpid

M3: supervisor — month-03/module-03-processes/processlab/processlab.c
SIGTERM and SIGKILL grace

M4: project specification — month-04/parallel-test-runner-starter/README.md
queue and worker architecture

M4: runner — month-04/parallel-test-runner-starter/ptrunner.c
pthreads, spawning, cleanup

M4: QA suite — month-04/parallel-test-runner-starter/tests/qa_runner.sh
regression, wraparound, stress

Official manuals and documentation

- Linux open / open file descriptions
- dup / dup2 shared offsets
- read and write return values

- poll and readiness
- termios serial settings
- Unix-domain sockets
- fork and exec
- execve program replacement
- waitpid and process statuses
- sigaction and signal dispositions
- async-signal-safe functions
- pthread_create / pthread_join
- pthread condition variables
- posix_spawn
- Linux kernel lock classes
- Linux kernel memory ordering
- ThreadSanitizer manual

> **ACCURACY NOTE**
> The PDF covers the main concepts and artifacts visibly present in Modules 1–4 on the audited GitHub branch, with additional explanatory examples and engineering recommendations labeled in context. It is not a claim that every extension or production-hardening measure was implemented in the repository.

Final rule: the tool will not save you if you do not know the invariant. Ask what state is shared, who owns it, who can block and how failure changes the cleanup path. That is the thread connecting your first four modules.